

Part 1: The Foundations (Ch 1, 2, 3)

- **Database Approach:** Centralized, self-describing, program-data independent. Solves file-based redundancy and data dependence.
- **ANSI-SPARC:** External (user view), Conceptual (logical blueprint), Internal (physical storage). Separating these provides **Data Independence**.
- **Architectures:**
 - **2-Tier:** Client handles logic/UI; Server handles DB.
 - **3-Tier:** Client (UI), App Server (Logic), DB Server (Data). Best for scalability.
 - **TP Monitor:** Middleware that manages transaction routing and connection pooling for OLTP efficiency.

Part 2: Relational Model & Algebra (Ch 4, 5)

- **Relational Rules:** Atomic values, unique rows, column/row order doesn't matter.
- **Keys:** Superkey (identifies row) $\rightarrow$ Candidate Key (minimal) $\rightarrow$ Primary Key (selected) $\rightarrow$ Foreign Key (links tables).
- **Integrity:** Entity Integrity (PK $\neq$ Null), Referential Integrity (FK must match parent or be Null).
- **Algebra:** Selection (σ : filter rows), Projection (π : filter columns), Join ($\bowtie$: combine tables), Division ($\div$: find "all").

Part 3: ER/EER Modeling (Ch 12, 13)

- **Building Blocks:** Entities (nouns), Attributes (adjectives), Relationships (verbs).
- **Traps:** Fix Fan Traps by rearranging, fix Chasm Traps by adding direct relationships.
- **EER:** Uses **Subclasses/Superclasses** (Inheritance).
 - *Disjoint (d):* An entity is only one subclass.
 - *Overlapping (o):* An entity can be multiple subclasses.
- **Weak Entities:** Do not have a PK; they depend on a parent entity (double-lined box).

Part 4: Normalization (Ch 14, 15)

- **1NF:** Atomic values.
- **2NF:** No partial dependencies (every non-key must depend on the *entire* PK).
- **3NF:** No transitive dependencies (A $\rightarrow$ B $\rightarrow$ C is illegal).
- **BCNF:** Every determinant must be a candidate key.
- **Anomalies:**
 - *Insertion:* Can't add data without a PK.
 - *Deletion:* Losing unrelated data when deleting a row.
 - *Modification:* Updating one fact across multiple rows.

Part 5: Transaction Management (Ch 22)

- **ACID:** Atomicity (all/nothing), Consistency (valid state), Isolation (hiding concurrency), Durability (permanent storage).

- **Concurrency Problems:**
 - *Lost Update:* One write overwrites another.
 - *Uncommitted Dependency (Dirty Read):* Reading data that will be rolled back.
- **Resolution:**
 - **2PL:** Growing phase (get locks), Shrinking phase (release locks). Guarantees serializability.
 - **Timestamping:** Older transactions get priority; no waiting, just abort/restart if conflict.
- **Recovery:** Use Write-Ahead Logs (WAL). **Deferred Update** (only REDO), **Immediate Update** (UNDO/REDO), **Shadow Paging** (no log, directory swap).

Part 6: Past Paper Solution Cheat Sheet

- **When you see "Find FDs":** Use the "Always Test"—pick a value from the left column and see if the right-side values always match.
- **When you see "Find PK":** Find the smallest set of columns that can "unlock" every other column in the table (The Master Key).
- **When you see "Anomalies":** Do not define them. **Apply them.** Pick a specific row from the table (e.g., "Deleting Patient John Smith") and explain what data is lost or corrupted.
- **When you see "Design an EER/ERD":**
 1. Extract Entities (Nouns).
 2. Extract Relationships (Verbs).
 3. Identify Attributes.
 4. Define Cardinality (1:1, 1:N, M:N).
 5. Always assume Primary Keys if not given (like ID or Name).

Final Advice for the Exam:

1. **Normalization:** Write the FDs first. If the FDs are right, the tables will split correctly automatically.
2. **Concurrency:** If asked about 2PL, remember: **Grow -> Shrink**. Never acquire a lock after releasing one.
3. **Diagrams:** Always include your assumptions (like "Assuming Primary Key is Name") so the examiner understands your logic even if they disagree with your choice.